COMPLETE SYSTEM ARCHITECTURE DOCUMENTATION

Tourist Recommender System

Overview

This document describes the **complete 3-layer architecture** for the Tourist Recommender System based on the research papers. The system uses modern web technologies and follows industry best practices for scalability and maintainability.

Architecture Layers

1. PRESENTATION LAYER (Frontend)

2. APPLICATION LAYER (Backend)

3. DATA LAYER (Database)

4. EXTERNAL SERVICES

5. INFRASTRUCTURE

---

1. PRESENTATION LAYER (Frontend)

Purpose

User-facing interfaces for tourists to interact with the system.

Components

A. Web Application

Technology Stack:

- **Framework**: React.js or Vue.js
- **UI Library**: Material-UI or Ant Design
- **Map Integration**: Google Maps JavaScript API
- **State Management**: Redux or Vuex
- **HTTP Client**: Axios

Features:

- User registration and login
- Trip planning forms (dates, budget, preferences)
- Interactive map showing POIs

- Itinerary timeline view
- POI details and photos
- Rating and feedback forms
- User dashboard

Pages:

1. Home/Landing page
2. User Registration/Login
3. Trip Planning Form
4. POI Browse and Search
5. Itinerary Display (with map)
6. User Profile and Preferences
7. Feedback and Rating

B. Mobile Application (Optional)

Technology Stack:

- **Framework**: React Native or Flutter
- **Features**: GPS, Camera, Push Notifications

Additional Mobile Features:

- Real-time location tracking
- Turn-by-turn navigation
- Offline map access
- Camera for POI photos
- Push notifications for itinerary updates

---

2. APPLICATION LAYER (Backend)

Purpose

Business logic, API endpoints, algorithm implementation, and service orchestration.

Technology Stack

Core Technologies:

- **Language**: Python 3.8+
- **Web Framework**: Flask or FastAPI

- **API Style**: RESTful API
- **Authentication**: JWT (JSON Web Tokens)
- **Task Queue**: Celery (for long-running algorithms)
- **Message Broker**: Redis or RabbitMQ

Libraries for Algorithms:

- **K-means Clustering**: scikit-learn
- **Genetic Algorithm**: DEAP or custom implementation
- **Distance Calculations**: geopy, numpy
- **Data Processing**: pandas, numpy

Components

A. API Gateway

Responsibilities:

- Route incoming requests to appropriate services
- Authentication and authorization (JWT validation)
- Rate limiting to prevent abuse
- Request/response logging
- CORS handling

Key Endpoints:

User Management:

```
POST   /api/auth/register
POST   /api/auth/login
GET    /api/auth/profile
PUT    /api/auth/profile
POST   /api/auth/preferences
```

POI Management:

```
GET    /api/pois
GET    /api/pois/{poi_id}
GET    /api/pois/search?query=...
GET    /api/pois/nearby?lat=...&lng=...
POST   /api/pois/{poi_id}/rate
```

Trip Management:

```
POST   /api/trips
GET    /api/trips
GET    /api/trips/{trip_id}
PUT    /api/trips/{trip_id}
DELETE /api/trips/{trip_id}
```

Itinerary Generation:

```
POST   /api/trips/{trip_id}/generate-itinerary
GET    /api/trips/{trip_id}/itinerary/{day}
POST   /api/itineraries/{itinerary_id}/feedback
```

B. User Service

Responsibilities:

- User registration and authentication
- Profile management
- Preference collection and storage
- Interest detection (Module I - NLC)

Functions:

```python
def register_user(email, password, name)
def login_user(email, password) -> jwt_token
def get_user_profile(user_id)
def update_preferences(user_id, preferences)
def detect_interests(user_id, text_input) # NLC
```

Module I Integration:

- Natural Language Classification (NLC)
- Can use Watson NLC API or custom ML model
- Analyzes user text to detect travel interests
- Updates user_interests table with confidence scores

C. POI Service

Responsibilities:

- Fetch POIs from Google Places API
- Process and cache POI data

- Analyze popularity (Module II - Google Reviews)
- Manage POI categories and opening hours

Functions:

```python
def fetch_pois(destination, category)
def get_poi_details(poi_id)
def calculate_popularity(poi_id) # Module II
def cache_poi_data(poi_data)
def get_nearby_pois(lat, lng, radius)
```

Module II Implementation:

```python
def analyze_popularity(poi_id):
    """
    Fetch reviews from Google Reviews API
    Calculate popularity score based on:
    - Number of reviews
    - Average rating
    - Recent review sentiment
    - Visit frequency
    """
    reviews = fetch_google_reviews(poi_id)
    sentiment_score = analyze_sentiment(reviews)
    popularity_score = calculate_weighted_score(
        rating=poi.rating,
        review_count=poi.user_ratings_total,
        sentiment=sentiment_score
    )
    return popularity_score
```

D. Trip Service

Responsibilities:

- CRUD operations for trips
- Manage trip constraints (dates, budget, schedule)
- Track trip status
- Coordinate with other services

Functions:

```python
```

```
def create_trip(user_id, destination, dates, constraints)
def get_user_trips(user_id)
def update_trip(trip_id, updates)
def delete_trip(trip_id)
def validate_constraints(trip_data)
```

E. Clustering Service (Module III)

Responsibilities:

- Implement K-means algorithm
- Group POIs by geographical proximity
- Create daily clusters
- Calculate centroids

Implementation:

python

```python
from sklearn.cluster import KMeans
import numpy as np

def cluster_pois_by_day(trip_id, selected_pois):
    """
    Module III: K-means Clustering
    Groups POIs into daily clusters
    """
    trip = get_trip(trip_id)
    n_clusters = trip.number_of_days

    # Extract POI coordinates
    coordinates = np.array([
        [poi.latitude, poi.longitude]
        for poi in selected_pois
    ])

    # Apply K-means
    kmeans = KMeans(n_clusters=n_clusters, random_state=42)
    cluster_labels = kmeans.fit_predict(coordinates)
    centroids = kmeans.cluster_centers_

    # Save to database
    for day in range(n_clusters):
        cluster = create_cluster(
            trip_id=trip_id,
            day_number=day + 1,
            centroid_lat=centroids[day][0],
            centroid_lng=centroids[day][1]
        )

        # Assign POIs to cluster
        day_pois = [poi for i, poi in enumerate(selected_pois)
                    if cluster_labels[i] == day]
        for poi in day_pois:
            add_poi_to_cluster(cluster.id, poi.id)

    return clusters
```

F. GA Optimization Service (Module IV)

Responsibilities:

- Implement Genetic Algorithm
- Solve TTDP/OPTW problem

- Optimize visit sequence
- Calculate fitness scores
- Respect time constraints

Implementation:

```python
```

```
from deap import base, creator, tools, algorithms
import random

def optimize_itinerary(cluster_id):
    """
    Module IV: Genetic Algorithm
    Optimizes POI visit sequence within a cluster
    """
    cluster = get_cluster(cluster_id)
    pois = get_cluster_pois(cluster_id)
    travel_times = get_travel_time_matrix(pois)

    # GA Setup
    creator.create("FitnessMax", base.Fitness, weights=(1.0,))
    creator.create("Individual", list, fitness=creator.FitnessMax)

    toolbox = base.Toolbox()
    toolbox.register("indices", random.sample, range(len(pois)), len(pois))
    toolbox.register("individual", tools.initIterate, creator.Individual,
                     toolbox.indices)
    toolbox.register("population", tools.initRepeat, list,
                     toolbox.individual)

    # Fitness function
    def evaluate_fitness(individual):
        """
        Calculate fitness score considering:
        - Total visit time
        - POI ratings
        - Opening hours compliance
        - Lunch time protection
        - Travel distance
        """
        total_score = 0
        current_time = cluster.trip.daily_start_time
        penalties = 0

        for i, poi_index in enumerate(individual):
            poi = pois[poi_index]

            # Check opening hours
            if not is_open(poi, current_time):
                penalties += 30  # 30 min penalty

            # Add POI score (rating * popularity)
            total_score += poi.rating * poi.popularity_score
```

```
        # Update time
        current_time += poi.expected_visit_duration

        # Travel to next POI
        if i < len(individual) - 1:
            next_poi_index = individual[i + 1]
            travel_time = travel_times[poi_index][next_poi_index]
            current_time += travel_time

        # Check lunch time invasion
        if invades_lunch_time(current_time, cluster.trip):
            penalties += 20

    fitness = total_score - penalties
    return (fitness,)

toolbox.register("evaluate", evaluate_fitness)
toolbox.register("mate", tools.cxOrdered)
toolbox.register("mutate", tools.mutShuffleIndexes, indpb=0.05)
toolbox.register("select", tools.selTournament, tournsize=3)

# Run GA
population = toolbox.population(n=100)
NGEN = 50

for gen in range(NGEN):
    offspring = toolbox.select(population, len(population))
    offspring = list(map(toolbox.clone, offspring))

    # Crossover
    for child1, child2 in zip(offspring[::2], offspring[1::2]):
        if random.random() < 0.7:
            toolbox.mate(child1, child2)
            del child1.fitness.values
            del child2.fitness.values

    # Mutation
    for mutant in offspring:
        if random.random() < 0.2:
            toolbox.mutate(mutant)
            del mutant.fitness.values

    # Evaluate
    invalid_ind = [ind for ind in offspring if not ind.fitness.valid]
    fitnesses = map(toolbox.evaluate, invalid_ind)
    for ind, fit in zip(invalid_ind, fitnesses):
```

```
            ind.fitness.values = fit

        population[:] = offspring

    # Get best solution
    best_individual = tools.selBest(population, 1)[0]

    # Save itinerary
    return create_itinerary_from_solution(
        cluster_id,
        best_individual,
        pois
    )
```

G. Recommendation Service

Responsibilities:

- Generate final itinerary
- Find restaurant suggestions for lunch
- Create alternative routes
- Format output for frontend

Functions:

```
python

def generate_complete_itinerary(trip_id)
def suggest_restaurants(itinerary_id, time)
def create_alternative_routes(cluster_id, n=3)
def format_itinerary_output(itinerary_id)
```

---

3. DATA LAYER

A. MySQL Database

Purpose: Primary relational database for all structured data

Table Groups:

1. User Management (3 tables)
2. POI Data (5 tables)
3. Trip Planning (2 tables)
4. Clustering (2 tables)
5. Itineraries (4 tables)

6. Feedback (2 tables)
7. System (1 table)

(See database_schema.sql for complete schema)

Database Configuration:

```python
# Production settings
MYSQL_HOST = 'localhost'
MYSQL_PORT = 3306
MYSQL_DATABASE = 'tourist_recommender'
MYSQL_USER = 'app_user'
MYSQL_PASSWORD = 'secure_password'

# Connection pool
MYSQL_POOL_SIZE = 10
MYSQL_MAX_OVERFLOW = 20
```

B. Redis Cache

Purpose: High-speed caching for API responses and session data

Cached Data:

- Google Places API responses (TTL: 24 hours)
- Google Reviews data (TTL: 12 hours)
- Distance Matrix results (TTL: 7 days)
- User sessions (TTL: session duration)
- Temporary GA results during optimization

Cache Keys:

```
places:{destination}:{category}
poi_details:{poi_id}
travel_time:{origin_id}:{destination_id}:{mode}
user_session:{session_id}
```

C. File Storage

Purpose: Store binary files (images, generated maps)

Options:

- **Development**: Local file system
- **Production**: Amazon S3, Google Cloud Storage, or Azure Blob Storage

File Structure:

```
/uploads
  /poi_photos
    /{poi_id}
      /photo1.jpg
      /photo2.jpg
  /user_uploads
    /{user_id}
      /profile.jpg
  /generated_maps
    /itinerary_{itinerary_id}.png
```

---

4. EXTERNAL SERVICES

A. Google Places API

Purpose: Fetch POI data including details, photos, and metadata

Endpoints Used:

- Place Search (Nearby, Text Search)
- Place Details
- Place Photos

Rate Limits:

- Free tier: Limited requests/day
- Paid tier: Higher limits

Cost Considerations:

- Cache responses aggressively
- Batch requests when possible
- Only fetch necessary fields

B. Google Reviews API

Purpose: Get user reviews and ratings for popularity analysis (Module II)

Data Retrieved:

- Review text
- Star ratings
- Review date
- User information

Processing:

- Sentiment analysis on review text
- Aggregate ratings
- Calculate popularity scores

C. Distance Matrix API

Purpose: Calculate travel times and distances between POIs

Parameters:

- Origins: POI coordinates
- Destinations: POI coordinates
- Mode: walking, driving, transit
- Units: metric

Optimization:

- Pre-calculate and store in database
- Update weekly or monthly
- Cache results

D. Google Maps JavaScript API

Purpose: Display interactive maps in frontend

Features Used:

- Map display
- Markers for POIs
- Routes/directions
- Info windows
- Custom markers

---

5. INFRASTRUCTURE & DEPLOYMENT

A. Web Server

Options:

- **Nginx** (recommended)
- **Apache**

Configuration:

nginx

```nginx
# Nginx config
server {
    listen 80;
    server_name touristrecommender.com;

    # Redirect to HTTPS
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl;
    server_name touristrecommender.com;

    ssl_certificate /path/to/cert.pem;
    ssl_certificate_key /path/to/key.pem;

    # Frontend static files
    location / {
        root /var/www/frontend/build;
        try_files $uri /index.html;
    }

    # API proxy
    location /api {
        proxy_pass http://localhost:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

B. Application Server

Technology:

- **Development**: Flask development server
- **Production**: Gunicorn or uWSGI

Configuration:

bash

```
# Gunicorn
gunicorn -w 4 -b 0.0.0.0:5000 app:app

# With gevent for async
gunicorn -k gevent -w 4 -b 0.0.0.0:5000 app:app
```

Containerization (Docker):

```
dockerfile

FROM python:3.9-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:5000", "app:app"]
```

C. Deployment Options

Option 1: Traditional VPS

- **Providers**: DigitalOcean, Linode, AWS EC2
- **Setup**:
  - Nginx as reverse proxy
  - Gunicorn for Python app
  - MySQL on same or separate server
  - Redis for caching
- **Cost**: $10-50/month

Option 2: Platform as a Service (PaaS)

- **Providers**: Heroku, Google App Engine, Azure App Service
- **Advantages**: Easy deployment, auto-scaling
- **Cost**: $25-100/month

Option 3: Containerized (Docker + Kubernetes)

- **Providers**: AWS ECS, Google Kubernetes Engine, Azure AKS

- **Advantages**: Scalability, microservices
- **Cost**: Variable, based on usage

Option 4: Serverless

- **Providers**: AWS Lambda, Google Cloud Functions
- **Advantages**: Pay per use, auto-scaling
- **Limitations**: Cold starts, execution time limits

D. Monitoring and Logging

Tools:

- **Application Monitoring**: New Relic, Datadog, or Prometheus
- **Log Management**: ELK Stack (Elasticsearch, Logstash, Kibana)
- **Error Tracking**: Sentry
- **Uptime Monitoring**: UptimeRobot, Pingdom

Metrics to Track:

- API response times
- Database query performance
- GA execution time
- Cache hit rates
- API error rates
- User activity

Data Flow Examples

Example 1: User Creates a Trip

```
1. User fills trip form on Web UI
   └─> POST /api/trips
2. API Gateway validates JWT
   └─> Routes to Trip Service
3. Trip Service validates data
   └─> Saves to MySQL (trips table)
4. Returns trip_id to frontend
5. Frontend redirects to trip details page
```

Example 2: Generate Itinerary (Complete Flow)

```
1. User clicks "Generate Itinerary"
```

└─> POST /api/trips/{trip_id}/generate-itinerary

2. User Service fetches user preferences
   └─> SELECT FROM user_preferences, user_interests

3. POI Service filters POIs
   └─> Based on preferences and destination
   └─> Calls Google Places API (if not cached)
   └─> Calls Google Reviews API for popularity
   └─> SELECT FROM pois WHERE ...

4. Clustering Service (Module III)
   └─> Groups POIs by day using K-means
   └─> INSERT INTO clusters, cluster_pois

5. GA Service (Module IV) - For each day:
   └─> Fetches travel_times matrix
   └─> Runs Genetic Algorithm
   └─> Optimizes visit sequence
   └─> INSERT INTO itineraries, itinerary_pois

6. Recommendation Service
   └─> Finds restaurants for lunch time
   └─> INSERT INTO itinerary_restaurants
   └─> Formats output

7. Returns complete itinerary to frontend
   └─> Frontend displays on map with timeline

Example 3: User Provides Feedback

1. User rates itinerary (P1-P16 questions)
   └─> POST /api/itineraries/{id}/feedback

2. API Gateway validates user
   └─> Routes to Recommendation Service

3. Saves feedback
   └─> INSERT INTO itinerary_feedback

4. Updates user interests (learning)
   └─> UPDATE user_interests
   └─> Adjust confidence scores

5. Returns success to frontend

Security Considerations

Authentication & Authorization

- **JWT tokens** with expiration
- **Password hashing** using bcrypt
- **HTTPS only** in production
- **Rate limiting** on API endpoints

API Security

- **Input validation** on all endpoints
- **SQL injection prevention** (parameterized queries)
- **XSS protection** (sanitize user input)
- **CORS policy** (whitelist frontend domain)

Data Privacy

- **Encrypt sensitive data** at rest
- **GDPR compliance** (user data deletion)
- **Secure API keys** (environment variables)

Scalability Considerations

Horizontal Scaling

- **Load balancer** (Nginx, HAProxy)
- **Multiple app servers**
- **Database replication** (master-slave)
- **Redis cluster**

Optimization Strategies

- **Database indexing** (see schema)
- **Query optimization** (avoid N+1 queries)
- **Caching strategy** (Redis)
- **Async processing** (Celery for GA)
- **CDN for static files**

Performance Targets

- **API response time**: < 200ms (95th percentile)

- **Itinerary generation**: < 30 seconds
- **Database queries**: < 50ms average
- **Cache hit rate**: > 80%

Development Workflow

Local Development Setup

```bash
# 1. Clone repository
git clone https://github.com/yourrepo/tourist-recommender.git
cd tourist-recommender

# 2. Backend setup
cd backend
python -m venv venv
source venv/bin/activate  # Windows: venv\Scripts\activate
pip install -r requirements.txt

# 3. Database setup
mysql -u root -p < schema.sql
python seed_data.py

# 4. Environment variables
cp .env.example .env
# Edit .env with your API keys

# 5. Run backend
flask run

# 6. Frontend setup (new terminal)
cd ../frontend
npm install
npm start
```

Testing

Backend Tests:

```bash
pytest tests/
pytest tests/test_clustering.py
pytest tests/test_ga.py
```

Frontend Tests:

```bash
npm test
npm run test:coverage
```

Cost Estimation (Monthly)

API Costs (Google)

- **Places API**: $0-200 (depending on usage)
- **Distance Matrix**: $0-100
- **Maps JavaScript API**: Free (< 28,000 loads/month)

Infrastructure

- **VPS/Cloud Server**: $10-50
- **Database**: Included or $10-20
- **Redis**: Included or $10
- **SSL Certificate**: Free (Let's Encrypt)

Total Estimated Cost

- **Minimal**: $20-50/month (low traffic)
- **Medium**: $100-300/month (moderate traffic)
- **High**: $500+/month (high traffic, scaling)

Future Enhancements

1. **Real-time collaboration** (multiple users planning together)
2. **Offline mode** for mobile app
3. **AR navigation** (augmented reality POI markers)
4. **Social features** (share itineraries, follow travelers)
5. **Multi-language support** (i18n)
6. **Weather integration** (adjust recommendations)
7. **Budget tracking** (actual vs. estimated costs)
8. **Booking integration** (hotels, restaurants)

Conclusion

This architecture provides a **complete, production-ready foundation** for your tourist recommender system. It follows industry best practices for:

- **Separation of concerns** (3-layer architecture)
- **Scalability** (horizontal scaling capabilities)
- **Maintainability** (modular services)
- **Security** (authentication, encryption)
- **Performance** (caching, optimization)

The system is designed to handle the requirements from the research paper while being practical to implement as an undergraduate final year project.